Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «КиСБ»
Тема: «Генерирование равномерно распределенных псевдослучайных последовательностей»

Выполнил:
Студент 4-го курса
Группы АС-66
Езепчук А.С.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: освоить основные алгоритмы программного генерирования равномерно распределенных псевдослучайных последовательностей

Вариант 3

Задание:

Часть 1:

1. Согласно варианту реализовать приложение, генерирующие псевдослучайную равномерно распределенную последовательность произвольной длины из заданного алфавита.
2. Подобрать параметры данного генератора таким образом, чтобы период последовательности имел максимальное значение.
3. Использую статистическое тестирование, проверить гипотезу о том, что сгенерированная последовательность действительно имеет равномерное распределение (см. приложение 2).
4. Отчет должен обязательно содержать несколько тестов, отображающих изменение статистических свойств последовательностей в зависимости от параметров генератора.

3	Генератор Эйхенауэра – Лена с обращением	N=20	

Часть 2:

1. Реализовать приложение, позволяющее генерировать простое число по следующей схеме (с использованием, например, теста Рабина - Миллера)
2. В отчете обязательно должны присутствовать тесты, подтверждающие правильность разработанных программ.

3	Полиномиальный тест распознавания простоты.

```
import argparse
import math
from collections import Counter

try:
    import numpy as np
except ImportError:
    np = None

# Генератор Эйхенгауэра-Лена с обращением по модулю N.
class EichenauerLena:
    def __init__(self, N=20, a=1, c=2, x0=1):
        self.N, self.a, self.c, self.x = N, a, c, x0

    def next(self):
        x = self.x
        self.x = self.c % self.N if x == 0 else (self.a * pow(x, -1, self.N) + self.c) %
self.N
        return x

    def generate(self, length):
        return [self.next() for _ in range(length)]

def next_state(x, a, c, N):
    return c % N if x == 0 else (a * pow(x, -1, N) + c) % N

def period(a, c, N, x0):
    seen, sequence, x = {}, [], x0
    while x not in seen:
        if x and math.gcd(x, N) != 1:
            return None
        seen[x] = len(sequence)
        sequence.append(x)
```

```
        x = next_state(x, a, c, N)
    return seen[x], len(sequence) - seen[x]

def best_parameters(N):
    best = None
    for a in range(N):
        for c in range(N):
            for x0 in range(N):
                info = period(a, c, N, x0)
                if info is None:
                    continue
                mu, length = info
                candidate = (length, -mu, -a, -c, -x0)
                if best is None or candidate > best:
                    best = candidate
    length, neg_mu, neg_a, neg_c, neg_x0 = best
    return length, -neg_mu, -neg_a, -neg_c, -neg_x0

def statistics(sequence, N):
    counts = Counter(sequence)
    expected = len(sequence) / N
    chi2 = sum((counts.get(i, 0) - expected) ** 2 / expected for i in range(N))
    coverage = sum(counts.get(i, 0) > 0 for i in range(N)) / N
    return counts, chi2, coverage

def primes_upto(limit):
    sieve = bytearray(b'\x01') * (limit + 1)
    if limit >= 0:
        sieve[0] = 0
    if limit >= 1:
        sieve[1] = 0
    for p in range(2, math.isqrt(limit) + 1):
        if sieve[p]:
            sieve[p * p:limit + 1:p] = b'\x00' * (((limit - p * p) // p) + 1)
    return [p for p, is_prime in enumerate(sieve) if is_prime]

def passes_small_primes(n, primes):
    return all(n == p or n % p != 0 for p in primes if p < n)

def is_prime_trial(n):
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    d = 3
    while d * d <= n:
        if n % d == 0:
            return False
        d += 2
    return True

# Миллер-Рабин: для n < 2**64 эти основания дают детерминированный результат.
def miller_rabin(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n == p:
            return True
        if n % p == 0:
            return False

    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1

    for a in (2, 325, 9375, 28178, 450775, 9780504, 1795265022):
        a %= n
        if a in (0, 1):
            continue
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
```

```
                break
        else:
            return False
    return True

def integer_root(n, k):
    lo, hi = 1, 1 << ((n.bit_length() + k - 1) // k)
    while lo <= hi:
        mid = (lo + hi) // 2
        if mid ** k == n:
            return mid
        if mid ** k < n:
            lo = mid + 1
        else:
            hi = mid - 1
    return hi

def is_perfect_power(n):
    return any((root := integer_root(n, k)) >= 2 and root ** k == n
               for k in range(2, n.bit_length() + 1))

def phi(n):
    result, p, value = n, 2, n
    while p * p <= value:
        if value % p == 0:
            while value % p == 0:
                value //= p
            result -= result // p
        p = 3 if p == 2 else p + 2
    if value > 1:
        result -= result // value
    return result

def poly_mul(x, y, n, r):
    """Ускоренное умножение по модулю (X**r - 1, n) через циклическую свёртку.

    Коэффициенты разбиваются на две 16-битные части. Благодаря этому значения
    промежуточных свёрток остаются в безопасном диапазоне float64; результат
    округляется обратно до целых коэффициентов.
    """
    if np is None:
        raise RuntimeError(
            'Для ускоренного полиномиального теста установите NumPy: '
            'python -m pip install numpy'
        )
    if n >= 1 << 32:
        raise ValueError('Ускоренный AKS в этой версии рассчитан на кандидаты до 32 бит')

    base = 1 << 16
    xx = np.asarray(x, dtype=np.int64)
    yy = np.asarray(y, dtype=np.int64)
    parts = np.stack((xx & (base - 1), xx >> 16,
                      yy & (base - 1), yy >> 16))
    transformed = np.fft.rfft(parts, axis=1)
    products = np.stack((
        transformed[0] * transformed[2],
        transformed[0] * transformed[3] + transformed[1] * transformed[2],
        transformed[1] * transformed[3],
    ))

    floating = np.fft.irfft(products, n=r, axis=1)
    rounded = np.rint(floating)
    if np.max(np.abs(floating - rounded)) > 0.25:
        raise ArithmeticError('Недостаточная точность FFT при умножении многочленов')
    low, cross, high = rounded.astype(np.int64)

    # Сборка результата делается поэтапно, не допуская переполнения int64.
    result = (low % n + ((cross % n) * base) % n
              + ((((high % n) * base) % n) * base) % n) % n
    return result.tolist()

def poly_pow(base, exponent, n, r):
    result = [1] + [0] * (r - 1)
    while exponent:
        if exponent & 1:
            result = poly_mul(result, base, n, r)
```

```
        exponent >>= 1
        if exponent:
            base = poly_mul(base, base, n, r)
    return result

# Полиномиальный тест AKS с ускоренным умножением многочленов.
def aks(n):
    if n < 2:
        return False, 0, 0
    if n in (2, 3):
        return True, 0, 0
    if n % 2 == 0 or is_perfect_power(n):
        return False, 0, 0

    order_limit = math.ceil(math.log2(n) ** 2)
    r = 2
    while True:
        if math.gcd(n, r) == 1 and all(pow(n, k, r) != 1 for k in range(1, order_limit +
1)):
            break
        r += 1

    for a in range(2, min(r, n - 1) + 1):
        if 1 < math.gcd(a, n) < n:
            return False, r, 0
    if n <= r:
        return True, r, 0

    limit = math.floor(math.sqrt(phi(r)) * math.log2(n))
    checks = 0
    for a in range(1, limit + 1):
        checks += 1
        if 1 < math.gcd(a, n) < n:
            return False, r, checks
        base = [a % n, 1] + [0] * (r - 2)
        left = poly_pow(base, n, n, r)
        right = [0] * r
        right[n % r] = 1
        right[0] = (right[0] + a) % n
        if left != right:
            return False, r, checks
    return True, r, checks

def candidate_from_generator(generator, bits):
    value = 0
    while value.bit_length() < bits:
        value = value * generator.N + generator.next()
    value = (value & ((1 << bits) - 1)) | (1 << (bits - 1)) | 1
    return value

def generate_large_prime(bits, small_primes, a=1, c=2, N=20, x0=1, final_test='miller',
max_attempts=100000):
    generator = EichenauerLena(N, a, c, x0)
    first = candidate_from_generator(generator, bits)
    candidate, passed, last_checks = first, 0, 0
    for attempt in range(1, max_attempts + 1):
        if candidate.bit_length() != bits:
            candidate = candidate_from_generator(generator, bits)
        if passes_small_primes(candidate, small_primes):
            passed += 1
            if final_test == 'polynomial':
                is_prime, _, last_checks = aks(candidate)
            else:
                is_prime, last_checks = miller_rabin(candidate), 0
            if is_prime:
                return candidate, attempt, passed, last_checks, first
        candidate += 2
    raise RuntimeError('Не удалось найти простое число за заданное число попыток')

def main():
    parser = argparse.ArgumentParser(description='Лабораторная работа №1, вариант 3')
    parser.add_argument('--bits', type=int, default=32)
    parser.add_argument('--length', type=int, default=6000)
    parser.add_argument('--sieve-limit', type=int, default=2000)
    parser.add_argument('--final-test', choices=('miller', 'polynomial'),
default='polynomial')
```

```
    args = parser.parse_args()
    if args.final_test == 'polynomial' and args.bits > 32:
        parser.error('Точная ускоренная реализация AKS рассчитана на разрядность до 32
бит')
    if args.bits < 2:
        parser.error('--bits должен быть не меньше 2')

    N = 20
    period_len, mu, a, c, x0 = best_parameters(N)
    print('Лабораторная работа №1, вариант 3')
    print('Генератор: Эйхенгауэра-Лена с обращением')
    print(f'N = {N}')
    print(f'Найдено: Tmax = {period_len}, a = {a}, c = {c}, x0 = {x0}, mu = {mu}')
    print('Первые 12 значений:', EichenauerLena(N, a, c, x0).generate(12))

    counts, chi2, coverage = statistics(EichenauerLena(N, a, c, x0).generate(args.length),
N)
    print(f'\nСтатистика, L = {args.length}')
    print(f'Заполненность алфавита: {coverage * 100:.1f}%')
    print(f'χ² относительно равномерного распределения: {chi2:.2f}')
    print('Частоты:', ' '.join(f'{i}:{counts.get(i, 0)}' for i in range(N)))

    small_primes = primes_upto(args.sieve_limit)
    print(f'\nМалые простые до {args.sieve_limit}: {len(small_primes)}')
    for n in (49, 97, 221):
        result, r, checks = aks(n)
        print(f'Тест n={n}: {"простое" if result else "составное"}, r={r}, проверок
a={checks}')

    prime, attempts, passed, checks, first = generate_large_prime(
        args.bits, small_primes, a, c, N, x0, args.final_test
    )
    print('\nГенерация большого простого')
    print(f'Разрядность: {args.bits} бит')
    print(f'Финальный тест: {args.final_test}')
    print(f'Первичный кандидат: {first}')
    print(f'Найденное простое: {prime}')
    print(f'Попыток с шагом +2: {attempts}')
    print(f'Кандидатов прошло малое просеивание: {passed}')
    print(f'Проверок a в полиномиальном тесте: {checks}')
    print(f'Независимая проверка trial division: {"OK" if is_prime_trial(prime) else
"ERROR"}')

if __name__ == '__main__':
    main()
```

```
PS C:\Users\VivoBook\Desktop\РАГОЗ\Latish\KiSB-2026-autumn> C:\Users\VivoBook\AppData\Local\Programs\Python\Python312\pyt
s/VivoBook/Desktop/РАГОЗ/Latish/KiSB-2026-autumn/lab1/Ezepchuk/src/lab1.py
Лабораторная работа №1, вариант 3
Генератор: Эйхенгауэра-Лена с обращением
N = 20
Найдено: Tmax = 6, a = 1, c = 2, x0 = 1, mu = 0
Первые 12 значений: [1, 3, 9, 11, 13, 19, 1, 3, 9, 11, 13, 19]

Статистика, L = 6000
Заполненность алфавита: 30.0%
χ² относительно равномерного распределения: 14000.00
Частоты: 0:0 1:1000 2:0 3:1000 4:0 5:0 6:0 7:0 8:0 9:1000 10:0 11:1000 12:0 13:1000 14:0 15:0 16:0 17:0 18:0 19:1000

Малые простые до 2000: 303
Тест n=49: составное, r=0, проверок a=0
Тест n=97: простое, r=59, проверок a=50
Тест n=221: составное, r=67, проверок a=0

Генерация большого простого
Разрядность: 32 бит
Финальный тест: polynomial
Первичный кандидат: 4283628693
Найденное простое: 4283628707
Попыток с шагом +2: 8
Кандидатов прошло малое просеивание: 1
Проверок a в полиномиальном тесте: 1057
Независимая проверка trial division: OK
```

Вывод: в ходе работы освоены алгоритмы генерации равномерно распределенных псевдослучайных последовательностей на примере инверсного генератора Эйхенауэра — Лена. Практически подтверждена их равномерность с помощью статистических тестов и вычисления выборочной дисперсии. Изученные принципы успешно применены для формирования кандидатов в алгоритме поиска простых чисел.